

Tugas 12: Dimensionality Reduction - PCA

Aisya Az Zahra - 0110224053

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: azzahrasyaa1705@gmail.com

Abstract. Penelitian ini menerapkan Principal Component Analysis (PCA) dan Support Vector Machine (SVM) pada dataset Breast Cancer dari Kaggle untuk melakukan klasifikasi jenis kanker payudara. Tahapan yang dilakukan meliputi eksplorasi awal data, pembersihan kolom kosong, pemisahan fitur dan label, encoding label, pembagian data menjadi train dan test, serta standardisasi fitur. Model baseline SVM pertama kali dibangun menggunakan seluruh 31 fitur asli dan menghasilkan akurasi sekitar 96% pada data uji. Selanjutnya dilakukan reduksi dimensi menggunakan PCA menjadi 3 komponen utama yang mampu menjelaskan sekitar 70% total variasi data, kemudian model SVM kedua dilatih menggunakan fitur hasil PCA dan tetap mencapai akurasi sekitar 95–96%. Visualisasi explained variance ratio dan scatter plot 3D PCA menunjukkan bahwa tiga komponen utama sudah cukup baik merepresentasikan struktur data dan memisahkan kelas benign dan malignant, sehingga PCA terbukti mampu menyederhanakan dimensi data medis tanpa menurunkan performa klasifikasi secara signifikan.

1.

```
[1]
✓ 1m      # Menghubungkan colab dengan google drive
          from google.colab import drive
          drive.mount('/content/gdrive')

          Mounted at /content/gdrive
```

Penjelasan :

Kode :

Kode diatas digunakan untuk menghubungkan google colab dan google drive.

Output :

Output menunjukan bahwa kode diatas berhasil dijalankan dan google colab telah terhubung dengan google drive, dan kita bias ke google colab melalui google drive.

2.

```
[2]
✓ 0s      # Membaca dataset lewat google drive
          path = path = "/content/gdrive/MyDrive/Praktikum02/Praktikum12/data/dataa.csv"
```

Penjelasan :

Kode ;

Kode diatas digunakan untuk membaca dataset yang akan digunakan pada praktikum 12 ini.

Output :

Output menunjukkan bahwa kode input berhasil dijalankan, data yang dimasukan berhasil dibaca dan siap untuk menjalankan kode selanjutnya.

```
[3]
✓ 3s

# Import library dasar
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # tetap diimport walau kadang tak terpakai

plt.rcParams['figure.figsize'] = (8, 6)
```

3.

Penjelasan :

Kode ini digunakan untuk menyiapkan semua library yang diperlukan dalam praktikum PCA dan klasifikasi SVM.

- Library numpy dan pandas dipakai untuk mengolah data secara numerik dan dalam bentuk tabel.
- Modul dari sklearn dipakai untuk membagi data train-test, melakukan standardisasi fitur, menerapkan PCA, membangun model SVM, serta menghitung metrik evaluasi seperti akurasi dan classification report.
- matplotlib dan Axes3D digunakan untuk membuat visualisasi 2D dan 3D.
- plt.rcParams['figure.figsize'] = (8, 6) mengatur ukuran default setiap grafik yang akan ditampilkan agar lebih rapi dan mudah dibaca.

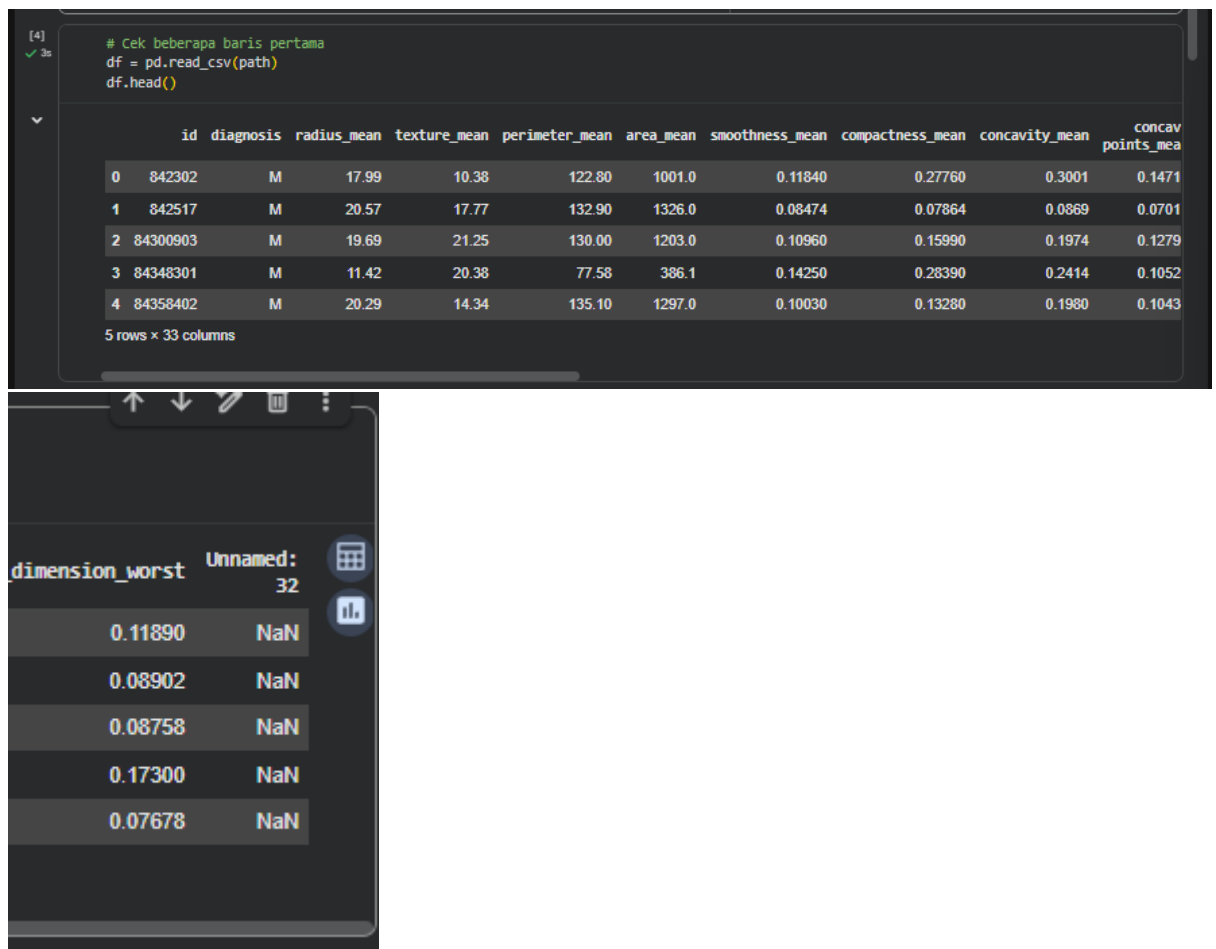
```
[4]
✓ 3s

# Cek beberapa baris pertama
df = pd.read_csv(path)
df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst	perimeter_worst	area_worst	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	fractal_dimension_worst	Unnamed: 32
0	842302	M	17.99	10.38	122.80	1001.0	0.11040	0.27780	0.3001	0.14710	...	17.33	104.00	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11090	NaN
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07064	0.0089	0.07017	...	23.41	158.00	1956.0	0.1238	0.1096	0.2416	0.1080	0.2750	0.08902	NaN
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08750	NaN
3	84340301	M	11.42	20.30	77.58	306.1	0.14250	0.20390	0.2414	0.10530	...	26.50	98.07	567.7	0.2098	0.0663	0.0889	0.2575	0.6638	0.17300	NaN
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13200	0.1900	0.10430	...	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07670	NaN

5 rows x 33 columns

4.



```
[4]
✓ 3s
# Cek beberapa baris pertama
df = pd.read_csv(path)
df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.1471
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.0701
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.1279
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.1052
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.1043

5 rows x 33 columns

dimension_worst	Unnamed: 32
0.11890	NaN
0.08902	NaN
0.08758	NaN
0.17300	NaN
0.07678	NaN

Penjelasan :

Kode :

- Kode `df = pd.read_csv(path)` digunakan untuk membaca file dataset Breast Cancer dalam format CSV dan menyimpannya ke dalam sebuah DataFrame bernama `df`.
- Perintah `df.head()` dipakai untuk menampilkan 5 baris pertama dari dataset, sehingga bisa dicek struktur kolom, tipe data, dan contoh isi datanya sebelum dilakukan pemrosesan lebih lanjut.

Output :

Output yang muncul adalah tabel berisi 5 baris pertama dengan total 33 kolom, di antaranya `id`, `diagnosis`, berbagai fitur statistik seperti `radius_mean`, `texture_mean`, sampai `fitur_worst`, serta satu kolom `Unnamed: 32` yang berisi nilai `NaN`. Informasi ini menunjukkan bahwa dataset memuat ID pasien, label diagnosis (M untuk malignant, B untuk benign), dan banyak fitur numerik hasil pengukuran medis; selain itu terlihat ada kolom kosong `Unnamed: 32` yang nantinya perlu dihapus karena tidak mengandung informasi.

```
[5]
✓ Os
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                     Non-Null Count  Dtype
---  ---
0   id                                           569 non-null    int64
1   diagnosis                                    569 non-null    object
2   radius_mean                                569 non-null    float64
3   texture_mean                               569 non-null    float64
4   perimeter_mean                             569 non-null    float64
5   area_mean                                  569 non-null    float64
6   smoothness_mean                            569 non-null    float64
7   compactness_mean                           569 non-null    float64
8   concavity_mean                             569 non-null    float64
9   concave points_mean                         569 non-null    float64
10  symmetry_mean                              569 non-null    float64
11  fractal_dimension_mean                     569 non-null    float64
12  radius_se                                  569 non-null    float64
13  texture_se                                 569 non-null    float64
14  perimeter_se                               569 non-null    float64
15  area_se                                    569 non-null    float64
16  smoothness_se                             569 non-null    float64
17  compactness_se                             569 non-null    float64
18  concavity_se                              569 non-null    float64
19  concave points_se                          569 non-null    float64
20  symmetry_se                               569 non-null    float64
21  fractal_dimension_se                       569 non-null    float64
22  radius_worst                              569 non-null    float64
23  texture_worst                             569 non-null    float64
24  perimeter_worst                           569 non-null    float64
25  area_worst                                569 non-null    float64
26  smoothness_worst                          569 non-null    float64
27  compactness_worst                         569 non-null    float64
28  concavity_worst                           569 non-null    float64
29  concave points_worst                      569 non-null    float64
30  symmetry_worst                            569 non-null    float64
31  fractal_dimension_worst                   569 non-null    float64
32  Unnamed: 32                               0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

5.

Penjelasan :

Kode :

Perintah `df.info()` digunakan untuk menampilkan informasi DataFrame, seperti jumlah baris, jumlah kolom, nama kolom, banyaknya nilai non-null, dan tipe data tiap kolom. Langkah kode ini penting untuk memastikan bahwa dataset sudah terbaca dengan benar, tidak ada kolom yang salah tipe, dan untuk mendeteksi apakah ada kolom yang berisi banyak nilai kosong sebelum lanjut ke tahap preprocessing.

Output :

Output menunjukkan bahwa dataset memiliki 569 baris dan 33 kolom, dengan 31 kolom bertipe numerik (float64), 1 kolom ID (int64), dan 1 kolom label diagnosis bertipe objek (kategori). Terlihat juga bahwa semua kolom fitur terisi penuh (569 non-null), sedangkan kolom Unnamed: 32 memiliki 0 non-null sehingga berisi nilai kosong seluruhnya; informasi ini menjadi dasar untuk menghapus kolom Unnamed: 32 pada tahap pembersihan data karena tidak memberikan informasi apapun.

```
[6]
✓ Os
df.describe()

count    5.690000e+02    569.000000    569.000000    569.000000    569.000000    569.000000    569.000000    569.000000    569.000000    569.000000    ...    569.000000    569.000000    569.000000    569.000000    569.000000    569.000000    569.000000    569.000000    569.000000
mean     3.037183e+07    14.127292    19.289649    91.969033    654.889104    0.096360    0.194341    0.089799    0.048919    0.101182    ...    25.677223    107.261213    880.583128    0.132369    0.254265    0.272188    0.114606    0.290076    0.06
std      1.250206e+08    3.524049    4.381036    24.298991    351.914129    0.014864    0.052813    0.079720    0.038803    0.027414    ...    6.148258    33.602542    568.356893    0.022832    0.157336    0.208624    0.065732    0.061067    0.01
min      6.670000e+03    6.961000    9.710000    43.790000    143.500000    0.052630    0.019330    0.000000    0.000000    0.106000    ...    12.020000    50.410000    105.200000    0.071170    0.027290    0.000000    0.000000    0.156500    0.05
25%     0.692180e+05    11.700000    16.170000    75.170000    420.300000    0.086370    0.064820    0.029560    0.020310    0.161500    ...    21.080000    84.110000    515.300000    0.116600    0.147200    0.114500    0.064830    0.250400    0.07
50%     0.600240e+05    13.370000    18.040000    86.240000    551.100000    0.095070    0.092630    0.061540    0.033590    0.176200    ...    25.410000    97.660000    606.500000    0.131300    0.211900    0.226700    0.089630    0.282200    0.08
75%     0.815129e+06    15.780000    21.800000    104.100000    782.700000    0.105300    0.130400    0.130700    0.074000    0.195700    ...    29.720000    125.400000    1084.000000    0.146000    0.339100    0.382900    0.161400    0.317900    0.09
max      8.113205e+08    28.110000    39.200000    188.500000    2501.000000    0.163400    0.345400    0.428800    0.201200    0.394000    ...    49.540000    251.200000    4254.000000    0.222600    1.050000    1.253000    0.291000    0.663000    0.20
0 rows x 32 columns
```

6.

[00]
✓ Os

df.describe()

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200

8 rows x 10 columns

Penjelasan :

Kode :

Perintah `df.describe()` digunakan untuk menampilkan ringkasan statistik deskriptif dari semua kolom numerik di dalam dataset. Fungsi ini secara otomatis menghitung nilai count, mean, standar deviasi, nilai minimum, kuartil (25%, 50%, 75%), dan nilai maksimum, sehingga membantu memahami sebaran dan skala setiap fitur sebelum dilakukan pemodelan.

Output :

Output menunjukkan statistik ringkas untuk setiap fitur numerik, seperti `radius_mean`, `texture_mean`, hingga fitur `_worst`, termasuk jumlah data (count), rata-rata (mean), sebaran (std), nilai terkecil (min), nilai tengah (50%), dan nilai terbesar (max); informasi ini membantu melihat apakah ada nilai ekstrem, skala antar fitur yang berbeda jauh, serta memastikan tidak ada fitur yang kosong selain `Unnamed: 32` yang selalu bernilai NaN. Kolom `Unnamed: 32` muncul dengan count 0 dan semua statistic-nya NaN, menegaskan bahwa kolom ini memang tidak berisi data dan layak dihapus pada tahap pembersihan data.

7.

```
[in] # Cek missing value
df.isnull().sum()

texture_mean 0
perimeter_mean 0
area_mean 0
smoothness_mean 0
compactness_mean 0
concavity_mean 0
concave points_mean 0
symmetry_mean 0
fractal_dimension_mean 0
radius_se 0
texture_se 0
perimeter_se 0
area_se 0
smoothness_se 0
compactness_se 0
concavity_se 0
concave points_se 0
symmetry_se 0
fractal_dimension_se 0
radius_worst 0
texture_worst 0
perimeter_worst 0
area_worst 0
smoothness_worst 0
compactness_worst 0
concavity_worst 0
concave points_worst 0
symmetry_worst 0
fractal_dimension_worst 0
Unnamed: 32 569
dtype: int64
```

Penjelasan :

Kode :

Perintah `df.isnull().sum()` digunakan untuk mengecek jumlah nilai kosong (NaN) di setiap kolom pada DataFrame. Fungsi `isnull()` akan menandai tiap sel yang kosong, lalu `sum()` menjumlahkannya per kolom sehingga kamu bisa mengetahui apakah ada fitur yang perlu ditangani sebelum analisis, misalnya dengan dihapus atau diimputasi.

Output :

Output menunjukkan bahwa semua kolom utama seperti `id`, `diagnosis`, dan seluruh fitur numerik memiliki 0 nilai kosong, artinya data pada fitur-fitur tersebut lengkap. Satu-satunya kolom yang memiliki 569 nilai kosong adalah `Unnamed: 32`, sehingga jelas kolom ini tidak berisi informasi apapun dan sebaiknya dihapus dari dataset agar tidak mengganggu proses standardisasi maupun PCA.

8.

```
[82]
✓ Os      # Cek data duplikat
          df.duplicated().sum()

np.int64(0)
```

Penjelasan :

Perintah `df.duplicated().sum()` digunakan untuk mengecek apakah ada baris data yang terduplikasi di dalam DataFrame. Fungsi `duplicated()` akan menandai baris yang isinya sama persis dengan baris sebelumnya, lalu `sum()` menghitung berapa banyak baris yang terdeteksi sebagai duplikat.

Output :

Nilai output 0 menunjukkan bahwa tidak ada satupun baris yang terduplikasi dalam dataset.

9.

```
[83]
✓ Os      # Eksplorasi label / target
          label_col = "diagnosis"

          print("\nDistribusi label:")
          print(df[label_col].value_counts())

Distribusi label:
diagnosis
B      357
M      212
Name: count, dtype: int64
```

Penjelasan :

Kode :

Kode ini menetapkan nama kolom target ke variabel `label_col` dengan nilai "diagnosis", lalu menampilkan distribusi nilai pada kolom tersebut menggunakan `value_counts()`. Tujuannya adalah untuk melihat berapa banyak sampel yang termasuk kelas benign (B) dan malignant (M), sehingga bisa diketahui apakah data seimbang atau cenderung tidak seimbang sebelum dibuat model klasifikasi.

Output :

Output menunjukkan bahwa terdapat 357 sampel dengan label B (benign) dan 212 sampel dengan label M (malignant). Artinya, dataset agak tidak seimbang karena kelas benign lebih banyak dibanding kelas malignant, tetapi perbedaannya masih dalam batas yang wajar sehingga model SVM masih bisa dilatih tanpa teknik penyeimbangan khusus.

10.

```
[84]
✓ Os      # Pemisahan fitur dan label
          X = df.drop(columns=[label_col, 'Unnamed: 32']) # drop 'Unnamed: 32' karena semua isinya NaN
          y = df[label_col]
```

Penjelasan :

Kode ini memisahkan data menjadi dua bagian, yaitu fitur dan label. Variabel `X` berisi semua kolom kecuali kolom target `diagnosis` dan kolom `Unnamed: 32` yang di-drop

karena seluruh isinya NaN dan tidak informatif, sehingga hanya fitur numerik yang relevan yang masuk ke dalam model. Variabel y menyimpan kolom diagnosis sebagai label yang akan diprediksi (benign atau malignant) oleh algoritma machine learning.

11.

```
[85]
✓ Os      # Encode label (misal B/M) menjadi 0/1 agar cocok untuk model ML
          from sklearn.preprocessing import LabelEncoder
          le = LabelEncoder()
          y_encoded = le.fit_transform(y)  # jadi 0/1
```

Penjelasan :

Kode ini menggunakan LabelEncoder untuk mengubah nilai kategori pada variabel target y (yang berisi label B dan M) menjadi bentuk numerik 0 dan 1. Pertama, objek LabelEncoder dibuat dan disimpan dalam variabel le, kemudian le.fit_transform(y) melakukan dua proses sekaligus: mempelajari kategori apa saja yang ada di y dan mengonversinya menjadi angka. Hasil konversi tersebut disimpan dalam variabel baru y_encoded yang siap digunakan sebagai target pada algoritma klasifikasi, karena sebagian besar model machine learning hanya bisa memproses label dalam bentuk angka, bukan teks.

12.

```
[86]
✓ Os      # Bagi data menjadi train dan test

          X_train, X_test, y_train, y_test = train_test_split(
              X,
              y_encoded,
              test_size=0.2,
              random_state=42,
              stratify=y_encoded
          )

          print("Shape X_train:", X_train.shape)
          print("Shape X_test :", X_test.shape)

          Shape X_train: (455, 31)
          Shape X_test : (114, 31)
```

Penjelasan :

Kode :

Kode ini membagi dataset menjadi dua bagian, yaitu data latih (train) dan data uji (test) menggunakan fungsi train_test_split. Variabel X dan y_encoded dipisah sehingga 80% data (karena test_size=0.2) dipakai untuk melatih model (X_train, y_train), sedangkan 20% sisanya dipakai untuk menguji performa model (X_test, y_test). Parameter random_state=42 menjaga pembagian data tetap konsisten setiap dijalankan, sedangkan stratify=y_encoded memastikan proporsi kelas benign dan malignant di train dan test tetap seimbang seperti di data asli.

Output :

Output menunjukkan bahwa X_train berisi 455 sampel dengan 31 fitur, sedangkan X_test berisi 114 sampel dengan 31 fitur yang sama. Artinya, dari total 569

baris data, 455 baris digunakan untuk proses training dan 114 baris untuk pengujian model; pembagian ini sesuai dengan rasio 80% train dan 20% test yang diatur pada parameter test_size.

```
[87]
✓ Os      # Standardisasi fitur (mean = 0, std = 1)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit + transform di data train
X_test_scaled = scaler.transform(X_test)      # hanya transform di data test

print("Contoh 3 baris X_train_scaled:")
print(X_train_scaled[:3])

▼
Contoh 3 baris X_train_scaled:
[[ -2.43221088e-01  5.18558727e-01  8.91825791e-01  4.24631702e-01
   3.83925436e-01 -9.74743706e-01 -6.89771505e-01 -6.88586446e-01
  -3.98175254e-01 -1.03915470e+00 -8.25056321e-01 -1.09317755e-01
  -5.59755400e-02 -2.10096206e-01 -1.59132582e-02 -1.00518399e+00
  -9.11941990e-01 -6.62815884e-01 -6.52561081e-01 -7.01889114e-01
  -2.75393571e-01  5.79797697e-01  1.31324246e+00  4.66908134e-01
   4.45982711e-01 -5.96154777e-01 -6.34722227e-01 -6.10227299e-01
  -2.35743918e-01  5.45663235e-02  2.18367276e-02]
 [ 4.08373367e-01 -5.16364088e-01 -1.63971029e+00 -5.41348716e-01
  -5.42961327e-01  4.76219058e-01 -6.31833818e-01 -6.04281166e-01
  -3.03074908e-01  5.21543093e-01 -4.54522896e-01 -6.04377961e-01
  -1.00104604e+00 -5.85429002e-01 -4.93453793e-01  4.03212009e-01
  -7.68173276e-01 -4.79187222e-01  1.14508478e-01 -1.42950761e-01
  -5.77397732e-01 -5.82458953e-01 -1.69029101e+00 -6.11934288e-01
  -5.87013537e-01  2.73581959e-01 -8.14844486e-01 -7.12666415e-01
  -3.23207881e-01 -1.37576237e-01 -9.04401643e-01]
 [ -2.42771402e-01 -3.68118388e-01  4.55514964e-01 -3.88249933e-01
  -4.02970039e-01 -1.43297929e+00 -3.83927370e-01 -3.42175070e-01
  -7.65459348e-01 -8.50857052e-01 -2.26170901e-01  3.03979894e-01
   1.05150064e+00 -1.69545176e-01 -8.09073190e-04 -3.10104091e-01
   1.10632991e+00  6.22584752e-01  2.73684564e-01  7.54482587e-01
   1.50810455e+00 -3.98622224e-01  1.81977388e-01 -4.75431283e-01
  -4.20778328e-01 -1.62278520e+00 -3.91399175e-01 -4.31312898e-01
  -8.90825037e-01 -6.75892999e-01 -1.44015592e-01]]
```

13.

Penjelasan :

Kode :

Kode ini melakukan standardisasi pada seluruh fitur numerik agar memiliki skala yang seragam, dengan rata-rata 0 dan standar deviasi 1. Objek StandardScaler() dibuat dan disimpan di variabel scaler, lalu fit_transform diterapkan pada X_train untuk menghitung parameter skala (mean dan standar deviasi tiap fitur) sekaligus mengubah data train menjadi bentuk terstandardisasi, sedangkan transform pada X_test hanya menggunakan parameter yang sudah dihitung dari data train agar tidak terjadi data leakage. Perintah print(X_train_scaled[:3]) menampilkan tiga baris pertama hasil standardisasi sebagai contoh.

Output :

Output yang terlihat berupa matriks berisi tiga baris dan 31 kolom dengan nilai-nilai float yang sudah berubah menjadi skala standar (banyak angkanya berada di sekitar 0, ada yang positif dan negatif). Setiap angka mewakili nilai fitur yang sudah dikurangi dengan rata-ratanya dan dibagi dengan standar deviasinya, sehingga tidak ada lagi fitur yang

dominasinya terlalu besar hanya karena satuan atau skala awalnya lebih besar; kondisi ini penting agar algoritma seperti PCA dan SVM dapat bekerja lebih stabil dan adil terhadap semua fitur.

```
[88]
✓ 0s

# Model baseline: SVM tanpa PCA

svm_nopca = SVC(kernel="rbf", gamma="scale")
svm_nopca.fit(X_train_scaled, y_train)

y_pred_nopca = svm_nopca.predict(X_test_scaled)

acc_nopca = accuracy_score(y_test, y_pred_nopca)
print("Akurasi SVM tanpa PCA :", acc_nopca)

print("\nClassification report (tanpa PCA):")
print(classification_report(y_test, y_pred_nopca))

print("Confusion matrix (tanpa PCA):")
print(confusion_matrix(y_test, y_pred_nopca))

Akurasi SVM tanpa PCA : 0.9649122807017544

Classification report (tanpa PCA):
              precision    recall  f1-score   support

     0       0.96       0.99       0.97        72
     1       0.97       0.93       0.95        42

 accuracy          0.96          114
  macro avg       0.97       0.96       0.96       114
 weighted avg     0.97       0.96       0.96       114

Confusion matrix (tanpa PCA):
[[71  1]
 [ 3 39]]
```

14.

Penjelasan :

Kode :

Kode ini membangun model klasifikasi dasar (baseline) menggunakan algoritma Support Vector Machine (SVM) tanpa reduksi dimensi PCA. Objek SVC dibuat dengan kernel RBF dan parameter `gamma="scale"`, lalu model dilatih menggunakan data latih yang sudah distandardisasi (`X_train_scaled, y_train`). Setelah itu, model digunakan untuk memprediksi label data uji (`y_pred_nopca = svm_nopca.predict(X_test_scaled)`), kemudian dihitung akurasi keseluruhan dengan `accuracy_score`, serta ditampilkan metrik lebih detail melalui `classification_report` dan `confusion_matrix` untuk melihat performa per kelas.

Output :

Nilai akurasi 0.9649 menunjukkan bahwa SVM tanpa PCA mampu mengklasifikasikan sekitar 96% data uji dengan benar. Tabel `classification report` memperlihatkan `precision` dan `recall` yang tinggi untuk kedua kelas:

- kelas 0 (misalnya benign) memiliki `precision` 0.96 dan `recall` 0.99,
- kelas 1 (malignant) memiliki `precision` 0.97 dan `recall` 0.93, yang berarti model cukup seimbang dalam mengenali kedua jenis kanker.

- Confusion matrix $\begin{bmatrix} 71 & 1 \\ 3 & 39 \end{bmatrix}$ menunjukkan bahwa dari 72 sampel kelas 0 hanya 1 yang salah diklasifikasi menjadi kelas 1, dan dari 42 sampel kelas 1 ada 3 yang salah diklasifikasi menjadi kelas 0, sehingga kesalahan prediksi relatif sedikit dibanding jumlah total sampel.

```
[89]
✓ Os      # PCA: reduksi dimensi menjadi 3 komponen utama

pca = PCA(n_components=3)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

explained = pca.explained_variance_ratio_
print("Explained variance ratio per komponen:", explained)
print("Total variansi yang dijelaskan (3 PC):", explained.sum())

Explained variance ratio per komponen: [0.43184634 0.17962517 0.0935514 ]
Total variansi yang dijelaskan (3 PC): 0.7050229101608617
```

15.

Penjelasan :

Kode :

Kode ini menerapkan Principal Component Analysis (PCA) untuk mereduksi dimensi fitur dari 31 fitur awal menjadi hanya 3 komponen utama. Objek PCA dibuat dengan `n_components=3`, kemudian `fit_transform` pada `X_train_scaled` menghitung arah komponen utama sekaligus memproyeksikan data train ke ruang baru berdimensi 3, sedangkan `transform` pada `X_test_scaled` hanya memproyeksikan data test dengan komponen yang sudah dipelajari dari data train. Variabel `explained` menyimpan nilai `explained_variance_ratio_`, yaitu proporsi variansi data asli yang berhasil dijelaskan oleh masing-masing komponen PCA.

Output :

Nilai `explained variance ratio per komponen`: `[0.43184634 0.17962517 0.0935514]` menunjukkan bahwa PC1 menjelaskan sekitar 43,18% variasi data, PC2 sekitar 17,96%, dan PC3 sekitar 9,36%. Total variansi yang dijelaskan oleh tiga komponen utama sebesar 0.7050 berarti sekitar 70,5% informasi dari semua fitur awal masih berhasil dipertahankan setelah reduksi dimensi menjadi 3 komponen, sehingga PCA mampu menyederhanakan data cukup banyak tanpa kehilangan terlalu banyak informasi penting.



16.

Penjelasan :

Kode :

Kode di gambar membuat visualisasi nilai `explained_variance_ratio_` dari objek PCA dalam bentuk diagram batang. Variabel `explained_var` diisi dengan proporsi variansi yang dijelaskan oleh masing-masing komponen utama, lalu `plt.bar([1, 2, 3], explained_var)` menggambar tiga batang untuk PC1, PC2, dan PC3. Pengaturan `xlabel`, `ylabel`, `title`, `xticks`, dan `ylim(0, 1)` digunakan untuk memberi keterangan sumbu, judul grafik, posisi label komponen, serta membatasi skala sumbu Y antara 0 dan 1 sehingga grafik mudah dibaca.

Output :

Output berupa bar chart dengan judul "Variansi yang Dijelaskan oleh 3 Komponen PCA", di mana sumbu X menunjukkan komponen utama 1, 2, dan 3, sedangkan sumbu Y menunjukkan nilai `explained variance ratio` masing-masing komponen. Tinggi batang pertama jauh lebih tinggi dibanding batang kedua dan ketiga, yang menggambarkan bahwa PC1 menyimpan informasi (variansi) data paling besar, diikuti PC2 dan PC3; grafik ini mendukung hasil numerik sebelumnya bahwa tiga komponen utama bersama-sama sudah mampu menjelaskan sekitar 70,5% variasi data.

17.

```
[91]
✓ Os

# Model SVM dengan fitur hasil PCA

svm_pca = SVC(kernel="rbf", gamma="scale")
svm_pca.fit(X_train_pca, y_train)

y_pred_pca = svm_pca.predict(X_test_pca)

acc_pca = accuracy_score(y_test, y_pred_pca)
print("Akurasi SVM dengan PCA :", acc_pca)

print("\nClassification report (dengan PCA):")
print(classification_report(y_test, y_pred_pca))

print("Confusion matrix (dengan PCA):")
print(confusion_matrix(y_test, y_pred_pca))

Akurasi SVM dengan PCA : 0.956140350877193

Classification report (dengan PCA):
              precision    recall  f1-score   support

     0       0.94      1.00      0.97        72
     1       1.00      0.88      0.94        42

 accuracy          0.96        114
 macro avg       0.97      0.94      0.95        114
weighted avg       0.96      0.96      0.96        114

Confusion matrix (dengan PCA):
[[72  0]
 [ 5 37]]
```

Penjelasan :

Kode :

Kode ini membangun kembali model SVM, tetapi kali ini menggunakan fitur yang sudah direduksi dengan PCA (X_train_pca dan X_test_pca) sebagai masukan. Objek SVC dengan kernel RBF dan gamma="scale" dilatih menggunakan svm_pca.fit(X_train_pca, y_train), lalu dipakai untuk memprediksi label data uji dalam ruang PCA melalui svm_pca.predict(X_test_pca). Setelah itu dihitung akurasi model dengan accuracy_score, dan performa per kelas dievaluasi lebih detail menggunakan classification_report serta confusion_matrix untuk melihat dampak penerapan PCA terhadap hasil klasifikasi.

Output :

Nilai akurasi 0.9561 menunjukkan bahwa SVM dengan fitur hasil PCA mampu mengklasifikasikan sekitar 95–96% data uji dengan benar, hanya sedikit di bawah akurasi model tanpa PCA, tetapi dengan jumlah fitur yang jauh lebih sedikit. Classification report memperlihatkan bahwa kelas 0 (benign) memiliki precision 0.94 dan recall 1.00, artinya hampir semua sampel benign dikenali dengan benar, sedangkan kelas 1 (malignant) memiliki precision 1.00 dan recall 0.88, sehingga masih ada sebagian kecil kanker ganas yang salah diklasifikasikan. Confusion matrix [[72, 0], [5, 37]] menunjukkan bahwa tidak ada sampel benign yang salah menjadi malignant, tetapi ada 5 sampel malignant yang salah terbaca sebagai benign; secara keseluruhan, hasil ini membuktikan bahwa PCA mampu mereduksi dimensi data sambil mempertahankan performa SVM yang tetap tinggi.

18.

```
[92]
✓ 14s # Visualisasi 3D PCA (PC1, PC2, PC3)

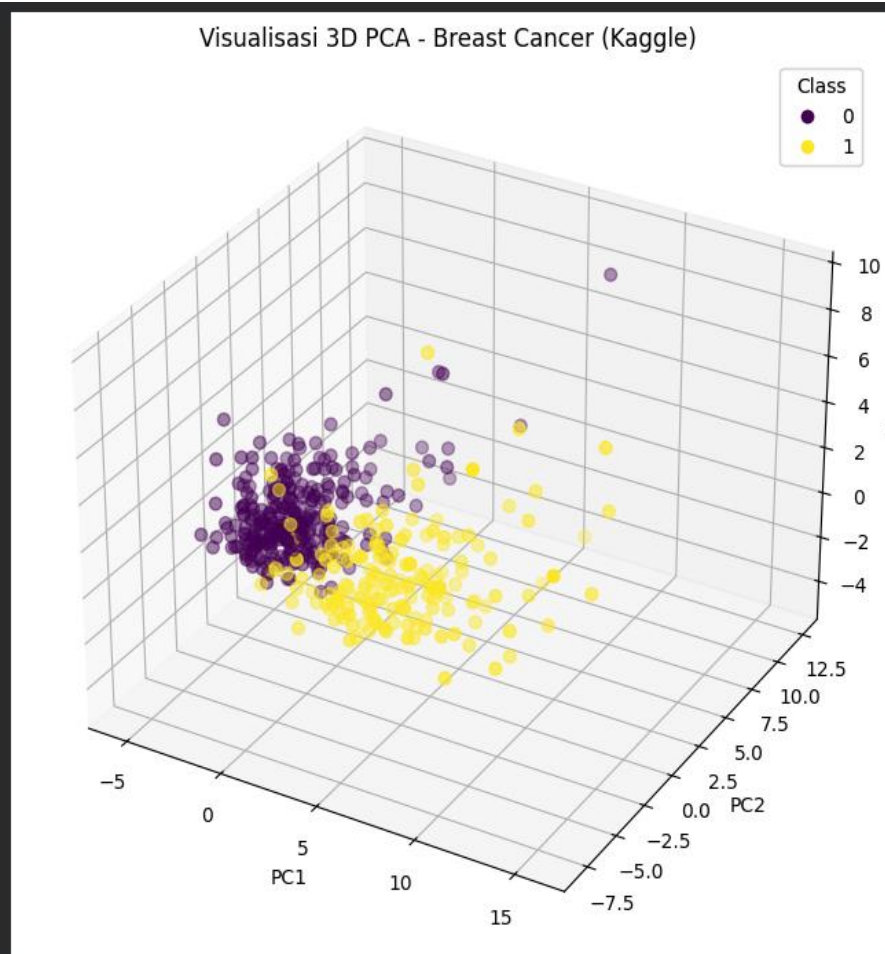
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection="3d")

scatter = ax.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    X_train_pca[:, 2],
    c=y_train,
    cmap="viridis",
    s=40
)

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title("Visualisasi 3D PCA - Breast Cancer (Kaggle)")

legend1 = ax.legend(*scatter.legend_elements(), title="Class")
ax.add_artist(legend1)

plt.show()
```



Penjelasan :

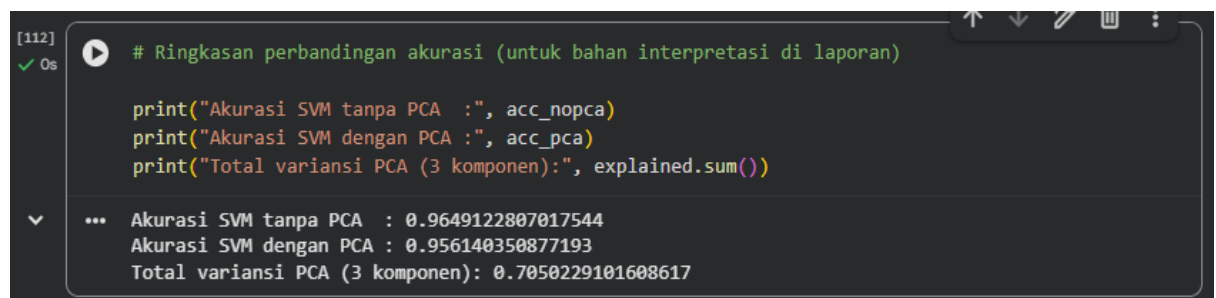
Kode :

Kode ini digunakan untuk memvisualisasikan hasil reduksi dimensi PCA dalam ruang tiga dimensi dengan menggunakan tiga komponen utama: PC1, PC2, dan PC3. Objek fig dan ax dibuat untuk menyiapkan kanvas dan sumbu 3D, kemudian fungsi ax.scatter menggambar titik-titik 3D dari data X_train_pca dengan koordinat [PC1, PC2, PC3] dan memberi warna berdasarkan kelas y_train menggunakan colormap "viridis". Label sumbu (set_xlabel, set_ylabel, set_zlabel) dan judul grafik (set_title) ditambahkan agar plot mudah dibaca, sedangkan legend_elements() digunakan untuk membuat legenda yang menjelaskan warna masing-masing kelas sebelum grafik ditampilkan dengan plt.show().

Output :

Output berupa scatter plot 3D yang menampilkan sebaran sampel Breast Cancer dalam ruang tiga komponen utama hasil PCA, di mana setiap titik mewakili satu pasien dan warna titik menunjukkan kelas diagnosis (misalnya kelas 0 dan 1 pada legenda). Terlihat bahwa titik-titik dengan warna berbeda cenderung membentuk dua kelompok yang cukup terpisah di sepanjang sumbu PC1–PC3, sehingga PCA mampu menangkap struktur data dan membantu memisahkan antara sampel benign dan malignant di ruang fitur berdimensi rendah.

19.



```
[112] # Ringkasan perbandingan akurasi (untuk bahan interpretasi di laporan)
✓ Os
print("Akurasi SVM tanpa PCA :", acc_nopca)
print("Akurasi SVM dengan PCA :", acc_pca)
print("Total variansi PCA (3 komponen):", explained.sum())

... Akurasi SVM tanpa PCA : 0.9649122807017544
Akurasi SVM dengan PCA : 0.956140350877193
Total variansi PCA (3 komponen): 0.7050229101608617
```

Penjelasan :

Kode :

Kode ini digunakan untuk menampilkan ringkasan hasil percobaan sehingga mudah dibandingkan di laporan. Baris pertama mencetak nilai akurasi model SVM yang dilatih menggunakan seluruh fitur asli tanpa reduksi dimensi, sedangkan baris kedua mencetak akurasi model SVM yang dilatih pada fitur hasil PCA dengan 3 komponen utama. Baris ketiga mencetak nilai total variansi yang berhasil dijelaskan oleh tiga komponen PCA (explained.sum()), yaitu seberapa besar informasi dari data asli yang masih dipertahankan setelah reduksi dimensi.

Output :

Output menunjukkan bahwa akurasi SVM tanpa PCA sebesar 0.9649 (sekitar 96,49%), sementara akurasi SVM dengan PCA sebesar 0.9561 (sekitar 95,61%); perbedaan akurasi ini sangat kecil, sehingga dapat disimpulkan bahwa PCA berhasil mengurangi jumlah fitur secara signifikan tanpa menurunkan performa model secara berarti. Nilai total variansi PCA 0.7050 berarti tiga komponen utama yang digunakan dalam model SVM sudah mampu menjelaskan sekitar 70,5% variasi data, sehingga mayoritas informasi penting dari 31 fitur awal tetap terwakili dalam ruang fitur berdimensi rendah.

Kesimpulan :

Kesimpulan dari praktikum ini adalah bahwa PCA efektif mereduksi jumlah fitur dari 31 menjadi 3 komponen utama sambil mempertahankan sekitar 70% informasi penting dari dataset Breast Cancer. Model SVM tanpa PCA menghasilkan akurasi kurang lebih 96,49%, sedangkan SVM dengan fitur hasil PCA menghasilkan akurasi sekitar 95,61%, sehingga perbedaan performa keduanya sangat kecil. Hal ini menunjukkan bahwa reduksi dimensi dengan PCA tidak mengorbankan kinerja model secara berarti, tetapi justru membuat representasi data lebih ringkas dan efisien secara komputasi. Nilai precision dan recall pada kedua model juga tinggi untuk kelas benign maupun malignant, yang berarti model mampu membedakan kedua jenis kanker dengan baik. Berdasarkan hasil tersebut, kombinasi PCA dan SVM dapat direkomendasikan sebagai pendekatan yang tepat untuk membangun sistem klasifikasi pada data medis berdimensi tinggi seperti kasus kanker payudara.

Link GitHub :

1. Praktikum Mandiri :

[Sem3ML Praktikum6/Praktikum12/Prakmandiri12.ipynb](#) at [main](#) · [Aisyaramli15/Sem3ML Praktikum6](#)

2. Praktikum bareng asdos :
[Sem3ML Praktikum6/Praktikum12/Prakbarengasdos12.ipynb](#) at [main](#) · [Aisyaramli15/Sem3ML Praktikum6](#)

3. Praktikum dari slide di kelas :
[Sem3ML Praktikum6/Praktikum12/12Prakdislide.ipynb](#) at [main](#) · [Aisyaramli15/Sem3ML Praktikum6](#)